

Code Sample

Short Version

Anonymous

This document contains excerpts from the replication code of a paper titled “Inequality in a Potential Equalizer of Opportunity: Variation in the Accessibility and Net Benefits of English-Medium Instruction in India.” Excerpts are followed by their corresponding outputs when appropriate. This project may have been updated since this file was generated; the most up-to-date versions of this document, the full paper, and the repository are available online for those interested. This PDF can be generated by running `make samples` or `scripts/run_full_build.sh` from the repo root.

Survey-Weighted Enrollment Model

The enrollment probit uses National Sample Survey (NSS) household weights, primary sampling units, and strata with a fixed set of child, household, and district covariates. The final analysis requires every variable used by the model and survey design, so incomplete inputs stop estimation instead of changing the specification.

File: `R/selection/estimate_selection_probit.R`

```
selection_probit_variables <- function(selection_data, require_all = FALSE) {  
  variables <- c(  
    "AGE", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "SECTOR",  
    "DIST_FROM_NEAREST_PRIMARY_CLASS", "father_educ",  
    "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",  
    "dmean_num_RECD_SCHOLARSHIP_STIPEND", "dmean_num_RECD_TXT_BOOKS",  
    "dmean_num_RECD_STATIONERY", "dmean_num_MID_DAY_MEAL_ETC_RECD",  
    "dmean_num_ENROLLMENT_COST"  
  )  
  
  missing <- setdiff(variables, names(selection_data))  
  if (isTRUE(require_all) && length(missing)) {
```

```

    stop(
      "Selection probit is missing required final analysis covariates: ",
      paste(missing, collapse = ", "),
      call. = FALSE
    )
  }
  variables[variables %in% names(selection_data)]
}

selection_survey_design_variables <- function() {
  c("FSU_SL_NO", "weight", "STATE", "SECTOR", "STRATUM", "SUB_STRATUM_NO")
}

## Retain the variables used by the selection model and survey design
##
## Earlier builds stored unrelated Block 5 schooling fields with the survey
## design. Editing one of those fields therefore recomputed the marginal effects
## even when the probit inputs were unchanged. Retain only variables used by the
## model and survey design.
project_selection_model_data <- function(selection_data) {
  selection_data <- safe_df(selection_data)
  design <- c("enrolled", selection_survey_design_variables())
  keep <- unique(c(design, selection_probit_variables(selection_data)))
  selection_data[intersect(keep, names(selection_data))]
}

## Estimate selection probit
##
estimate_selection_probit <- function(selection_data, cfg) {
  if (!"enrolled" %in% names(selection_data) || all(is.na(selection_data$enrolled))) {
    return(list(status = "out_of_active_pipeline", reason = "No enrolled variable."))
  }

  final_mode <- is_final_mode(cfg)
  covars <- selection_probit_variables(selection_data, require_all = final_mode)
  if (!length(covars)) {
    return(list(status = "out_of_active_pipeline", reason = "No probit covariates."))
  }
  f_probit <- stats::reformulate(covars, response = "enrolled")

  if (final_mode) {

```

```

    if (!requireNamespace("survey", quietly = TRUE)) {
      stop("The final selection probit requires the survey package.", call. = FALSE)
    }

    design <- build_survey_design_selection(selection_data, require_all = TRUE)
    out <- with_survey_lonely_psu(
      fit_selection_probit(design, f_probit)
    )
    return(stabilize_selection_model_formula(out, f_probit))
  }

  out <- stats::glm(
    f_probit,
    data = selection_data,
    family = stats::binomial(link = "probit")
  )
  stabilize_selection_model_formula(out, f_probit)
}

#' Store the fitted selection formula inside the saved model call
#'
#' Programmatic fitting originally left the local symbol `f_probit` in the saved
#' call. After {targets} serialized the model, packages reconstructing model data
#' could no longer resolve that local name. Store the formula itself in the call
#' and keep the formula attribute used by downstream validation.
stabilize_selection_model_formula <- function(model, formula) {
  if (!inherits(formula, "formula")) {
    stop("Selection model formula must inherit from formula.", call. = FALSE)
  }
  if (!is.null(model$call)) model$call$formula <- formula
  attr(model, "selection_probit_formula") <- formula
  model
}

#' Build survey design selection
#'
build_survey_design_selection <- function(selection_df, require_all = FALSE) {
  required <- selection_survey_design_variables()
  missing <- setdiff(required, names(selection_df))
  if (isTRUE(require_all) && length(missing)) {
    stop(
      "Selection survey design is missing required final analysis fields: ",

```

```

    paste(missing, collapse = ", "),
    call. = FALSE
  )
}

psu <- first_col(selection_df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
weight <- first_col(selection_df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
strata_cols <- intersect(c("STATE", "SECTOR", "STRATUM", "SUB_STRATUM_NO"), names(selection_df))
if (is.null(psu) || is.null(weight) || !length(strata_cols)) {
  if (isTRUE(require_all)) {
    stop("The final selection survey design could not be constructed.", call. = FALSE)
  }
  return(NULL)
}
selection_df$.survey_strata <- interaction(selection_df[strata_cols], drop = TRUE)
survey::svydesign(
  ids = stats::as.formula(paste0("~", psu)),
  strata = ~.survey_strata,
  weights = stats::as.formula(paste0("~", weight)),
  data = selection_df,
  nest = TRUE
)
}

#' Fit selection probit
#'
fit_selection_probit <- function(selection_design, f_probit) {
  survey::svyglm(
    f_probit,
    design = selection_design,
    family = stats::quasibinomial(link = "probit")
  )
}

```

Average Marginal Effects on the Estimation Sample

Average marginal effects are computed with `marginaleffects::avg_slopes()` over the observations used to fit the probit, weighted by their NSS sampling weights. The same calculation produces the enrollment table below.

File: R/selection/compute_average_marginal_effects.R

```

#' extract the fitted sample and survey weights used to average marginal effects
#'
ame_model_data_and_weights <- function(model) {
  model_data <- as.data.frame(stats::model.frame(model))

  model_weights <- if (inherits(model, "svyglm") && !is.null(model$survey.design)) {
    stats::weights(model$survey.design, type = "sampling")
  } else {
    tryCatch(stats::weights(model, type = "prior"), error = function(e) NULL)
  }

  if (is.null(model_weights)) model_weights <- rep(1, nrow(model_data))
  model_weights <- as.numeric(model_weights)
  if (length(model_weights) != nrow(model_data)) {
    stop("AME averaging weights do not align with the estimation sample.", call. = FALSE)
  }
  if (any(!is.finite(model_weights)) || any(model_weights < 0) || sum(model_weights) <= 0) {
    stop("AME averaging weights must be finite, nonnegative, and have positive total weight.", call. = FALSE)
  }

  list(data = model_data, wts = model_weights)
}

#' Run the standard marginaleffects implementation
#'
run_avg_slopes <- function(model, newdata = NULL, wts = TRUE, numberiv = NULL) {
  args <- list(
    model = model,
    newdata = newdata,
    wts = wts,
    vcov = TRUE,
    type = "response"
  )
  if (!is.null(numberiv)) args$numberiv <- numberiv
  do.call(marginaleffects::avg_slopes, args)
}

#' Compute AMEs with marginaleffects
#'
compute_ames_marginaleffects <- function(model) {
  # Earlier calls relied on marginaleffects to recover weights from serialized

```

```

# svyglm objects. Some saved models no longer exposed those weights reliably,
# so pass the fitted model frame and sampling weights explicitly. This keeps the
# averaging population equal to the survey estimation sample. `marginaleffects`
# uses the model's design-based covariance matrix for uncertainty.

amed <- ame_model_data_and_weights(model)
run_avg_slopes(model, newdata = amed$data, wts = amed$wts)
}

```

Table 7: Average Marginal Effects for School Enrollment

Term	Enrolled (1 = yes)
Age (years)	-0.028*** (0.000)
Female (ref: Male)	0.044*** (0.003)
Household size	-0.004*** (0.001)
Religion: Muslim (ref: Hindu)	-0.126*** (0.007)
Religion: Christian	-0.043*** (0.013)
Religion: Sikh	0.040 (0.026)
Religion: Jain	-0.001 (0.012)
Religion: Buddhist	0.047* (0.024)
Religion: Zoroastrian	0.101*** (0.034)
Religion: Other	0.252*** (0.003)
Social group: Scheduled Tribe (ref: Other)	-0.102*** (0.008)
Social group: Scheduled Caste	-0.075*** (0.006)
Social group: Other Backward Class	-0.029*** (0.005)
Urban (ref: Rural)	-0.003 (0.005)
Distance 1–2km (ref: <1km)	-0.077*** (0.026)
Distance 2–3km	-0.001 (0.010)
Distance 3–5km	0.004 (0.066)
Distance > 5km	0.036 (0.035)
Male household education: Literate, no school (ref: Illiterate)	0.097*** (0.017)
Male household education: Literate, school < primary	0.127*** (0.006)
Male household education: Primary	0.142*** (0.005)

Table 7: Average Marginal Effects for School Enrollment (*continued*)

Term	Enrolled (1 = yes)
Male household education: Upper primary	0.195*** (0.006)
Male household education: Secondary	0.212*** (0.007)
Male household education: Higher secondary	0.247*** (0.008)
Male household education: Postsecondary+	0.245*** (0.009)

* $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$. Entries are average changes in predicted enrollment probability. Continuous covariates are average slopes; categorical covariates are average discrete comparisons against the stated reference category, evaluated over the observed covariate distribution. Standard errors use the NSS sampling design, including household weights, primary sampling units, and strata formed from state, stratum, and substratum identifiers. The model also controls for district averages of schooling costs and benefits among enrolled children; the table displays marginal effects for child and household covariates. Source: NSS 64th Round, 2007–08, Participation and Expenditure in Education.

Assessing Weak-Instrument Strength with the Montiel Olea–Pflueger Effective F

The fitted instrumental-variable model is passed to `momentfit::MOPtest()`, using state clusters when they are available, to obtain the Montiel Olea–Pflueger effective F, critical value, p-value, and effective degrees of freedom. The table below also contains two-stage least squares (2SLS) and Anderson–Rubin results produced by the additional implementation files linked immediately before the table.

File: `R/iv/weak_identification.R`

```
mop_effective_f <- function(
  model, analysis_data, tau = 0.10, size = 0.05) {
  unavailable <- function(reason) {
    list(
      statistic = NA_real_, critical_value = NA_real_, p.value = NA_real_,
      effective_df = NA_real_, tau = tau, size = size,
      status = "not_estimated", reason = reason
    )
  }

  if (!inherits(model, "ivreg")) {
    return(unavailable("Montiel Olea-Pflueger effective F requires an ivreg model."))
  }

  if (!requireNamespace("momentfit", quietly = TRUE)) {
    return(unavailable("Package 'momentfit' is not installed."))
  }
}
```

```

if (!is.finite(tau) || tau <= 0 || !is.finite(size) || size <= 0 || size >= 1) {
  return(unavailable("Montiel Olea-Pflueger tau and size must be valid probabilities."))
}

endogenous <- suppressWarnings(as.integer(model$endogenous))
if (length(endogenous) != 1L || !is.finite(endogenous)) {
  return(unavailable("Montiel Olea-Pflueger effective F requires exactly one endogenous regressor."))
}

regression_formula <- tryCatch(
  stats::formula(model, component = "regressors"),
  error = function(e) NULL
)
instrument_formula <- tryCatch(
  stats::formula(stats::delete.response(stats::terms(model, component = "instruments")),
  error = function(e) NULL
)
if (is.null(regression_formula) || is.null(instrument_formula)) {
  return(unavailable("Could not recover ivreg component formulas for Montiel Olea-Pflueger effective F."))
}
needed <- unique(c(all.vars(regression_formula), all.vars(instrument_formula)))
data <- iv_analysis_frame(analysis_data, needed)
missing <- setdiff(needed, names(data))
if (length(missing)) {
  return(unavailable(paste(
    "Missing variables for Montiel Olea-Pflueger effective F:",
    paste(missing, collapse = ", ")
  )))
}
moment_data <- data[needed]
if (any(!stats::complete.cases(moment_data))) {
  return(unavailable("Montiel Olea-Pflueger analysis data contain incomplete model rows."))
}

stored_cluster <- attr(model, "cluster_state", exact = TRUE)
cluster <- if (
  length(stored_cluster) == stats::nobs(model) &&
  !anyNA(stored_cluster) && length(unique(stored_cluster)) >= 2L
) {
  as.vector(stored_cluster)
} else {

```

```

    iv_model_cluster(model, data)
  }
covariance <- if (is.null(cluster)) "MDS" else "CL"
covariance_options <- if (is.null(cluster)) {
  list(type = "HCO")
} else {
  # The momentfit::MOPtest() function uses an HCO clustered moment covariance with its own
  # finite-cluster adjustment. The conventional first-stage Wald F is computed
  # separately with the HC1 covariance used for the first-stage regression.
  list(
    cluster = data.frame(cluster = cluster),
    type = "HCO", cadjust = TRUE, multi0 = FALSE
  )
}

result <- tryCatch({
  moment_model <- momentfit::momentModel(
    regression_formula,
    instrument_formula,
    data = moment_data,
    vcov = covariance,
    vcovOptions = covariance_options
  )
  momentfit::MOPtest(
    moment_model,
    tau = tau,
    size = size,
    estMethod = "TSLS",
    simplified = TRUE,
    print = FALSE
  )
}, error = function(e) e)
if (inherits(result, "error")) return(unavailable(conditionMessage(result)))

values <- suppressWarnings(as.numeric(result[c("Feff", "critValue", "pvalue", "Keff"])))
names(values) <- c("statistic", "critical_value", "p.value", "effective_df")
if (length(values) != 4L || any(!is.finite(values))) {
  return(unavailable("momentfit::MOPtest() did not return finite effective-F statistics."))
}

c(

```

```
as.list(values),  
list(tau = tau, size = size, status = "estimated", reason = NA_character_)  
)  
}
```

Additional code used for this table: R/iv/estimate_2sls.R, R/iv/consumption_outcome_registry.R,
R/output/appendix_identification_exhibits.R, R/output/save_tables.R.

Table 10: Inference with Weak Instruments for Long-Run Consumption Change

	Real log consumption change, 2004-05 to 2022-23	Real log consumption change, 2004-05 to 2023-24
EMI exposure	0.117 (0.182)	0.137 (0.281)
MOP effective F	0.69	0.41
AR p -value at zero	0.029	0.081
AR 95% accepted set	$[-0.193, -0.030]^{\dagger} \cup [0.009, 0.193]^{\dagger}$	$[-0.186, 0.186]^{\dagger}$
Direct effect threshold for $\beta = 0$ (share of reduced form)	16.6%	0.0%
Observations	524	522

The table reports conventional 2SLS coefficients with state-clustered standard errors. Instrument strength is summarized by the Montiel Olea–Pflueger effective F statistic, and Anderson–Rubin tests provide inference robust to weak instruments. The 95% AR accepted sets are displayed as unions of closed intervals. A dagger marks a component that reaches an evaluated search boundary. Its outer endpoint is the numerical search limit and should not be interpreted as a finite confidence bound. The direct effect threshold is the smallest permitted direct effect, expressed as a share of the absolute reduced form, for which the AR set includes $\beta = 0$. Sources: Census of India 2001; NSS 61st Round, 2004–05; NSS 64th Round, 2007–08; HCES 2022–23; HCES 2023–24.

Mapping District Estimates on 2001 Census Boundaries

Each statistic is joined to the complete 2001 Census district boundary layer, leaving districts outside a particular estimation sample visible as missing data. Disputed territorial polygons are shown as no data, state outlines are reconstructed from the complete boundary layer, and the same functions draw categorical or numeric choropleths, including diverging scales.

File: R/output/save_figures.R

```
build_public_ggplot_map <- function(plot_data, spec, boundary_reference = NULL) {
  need_pkg("ggplot2", "classified choropleth maps")
  style <- public_map_style(spec$variable)
  fill <- public_map_fill(plot_data, spec$variable, style)
  plot_data <- fill$data
  overlay <- plot_data[map_overlay_rows(plot_data, fill$fill), , drop = FALSE]
  if (!nrow(overlay)) {
    stop("Map figure '", spec$name, "' has no non-missing overlay districts for variable '", spec$variable,
      "\n↳ '", call. = FALSE)
  }

  disputed_display <- public_map_disputed_display(plot_data, boundary_reference)
  display_data <- mask_public_map_disputed_areas(plot_data, disputed_display)
```

```

state_outlines <- public_map_state_outlines(display_data)
base <- ggplot2::ggplot() +
  ggplot2::geom_sf(
    data = display_data, ggplot2::aes(fill = .data[[fill$fill]]),
    color = "grey55", linewidth = map_district_boundary_linewidth(),
    linetype = "solid"
  )
if (!is.null(disputed_display) && nrow(disputed_display)) {
  base <- base + ggplot2::geom_sf(
    data = disputed_display, fill = map_disputed_no_data_colour(),
    color = "grey45", linewidth = map_disputed_boundary_linewidth(),
    linetype = "solid"
  )
}
if (!is.null(state_outlines) && nrow(state_outlines)) {
  base <- base + ggplot2::geom_sf(
    data = state_outlines, color = "grey15",
    linewidth = map_major_boundary_linewidth(), linetype = "solid"
  )
}
base <- base +
  ggplot2::coord_sf(datum = NA) +
  ggplot2::labs(fill = fill$title) +
  ggplot2::theme_void(base_size = 10) +
  ggplot2::theme(
    legend.position = "right",
    legend.title = ggplot2::element_text(size = 12, face = "bold"),
    legend.text = ggplot2::element_text(size = 10),
    legend.key.height = grid::unit(16, "pt"),
    legend.spacing.y = grid::unit(4, "pt"),
    plot.margin = grid::unit(c(2, 2, 2, 2), "pt")
  )

if (isTRUE(fill$continuous)) {
  return(base + ggplot2::scale_fill_gradientn(
    colours = fill$colors,
    limits = fill$limits,
    na.value = map_no_data_colour(),
    guide = public_map_colorbar_guide()
  ))
}

```

```

base +
  ggplot2::scale_fill_manual(
    values = fill$colors,
    breaks = names(fill$colors),
    limits = names(fill$colors),
    drop = FALSE,
    na.translate = TRUE,
    na.value = map_no_data_colour()
  ) +
  ggplot2::guides(
    fill = ggplot2::guide_legend(
      override.aes = map_legend_override(fill$colors)
    )
  )
}

complete_public_map_geometry <- function(district_panel, map_geometry) {
  if (!has_sf_geometry(map_geometry)) {
    stop("District maps require the complete 2001 Census district geometry.", call. = FALSE)
  }
  key <- "target_unit_2001"
  if (!key %in% names(map_geometry) && "unit_id" %in% names(map_geometry)) {
    map_geometry[[key]] <- map_geometry$unit_id
  }
  if (!key %in% names(map_geometry) || !key %in% names(district_panel)) {
    stop("Map geometry and analysis data must contain a 2001 Census district key.", call. = FALSE)
  }
  if (anyDuplicated(map_geometry[[key]])) {
    stop("Complete 2001 Census map geometry must contain one row per district.", call. = FALSE)
  }
  attributes <- if (inherits(district_panel, "sf")) sf::st_drop_geometry(district_panel) else
  ↪ safe_df(district_panel)
  attributes <- attributes[!duplicated(attributes[[key]]), , drop = FALSE]
  out <- merge(map_geometry, attributes, by = key, all.x = TRUE, sort = FALSE)

  # Derive state identity from the complete 2001 Census boundary identifier so
  # state and national outlines include districts with missing analysis values.
  # The district identifier stores the 2001 state code directly.
  boundary_state <- public_map_state_code_2001(out[[key]])
  if ("state_code_2001" %in% names(out)) {

```

```

panel_state <- plain_chr(out$state_code_2001)
mismatch <- !is.na(panel_state) & nzchar(panel_state) & panel_state != boundary_state
if (any(mismatch)) {
  stop("Analysis state codes disagree with the state codes in the 2001 Census district identifiers.",
    ↪ call. = FALSE)
}
}
out$state_code_2001 <- boundary_state
out
}

save_map_plot_formats <- function(map_plot, path_base, formats, width = 8, height = 6, dpi = 300) {
  save_plot_formats(map_plot, path_base, formats, width = width, height = height, dpi = dpi)
}

save_map_figure <- function(spec, path_base, district_panel, map_geometry, boundary_reference, formats) {
  if (!has_sf_geometry(district_panel)) {
    stop("Map figure '", spec$name, "' requires an sf district_panel with validated geometry.", call. =
      ↪ FALSE)
  }
  if (is.null(spec$variable) || !spec$variable %in% names(district_panel)) {
    stop("Map figure '", spec$name, "' is missing variable '", spec$variable, "'.", call. = FALSE)
  }

  plot_data <- complete_public_map_geometry(district_panel, map_geometry)
  plot_data <- prepare_public_map_data(plot_data, spec$variable)
  p <- build_public_ggplot_map(plot_data, spec, boundary_reference = boundary_reference)
  save_map_plot_formats(p, path_base, formats, width = 7.2, height = 5.2, dpi = 300)
}

```

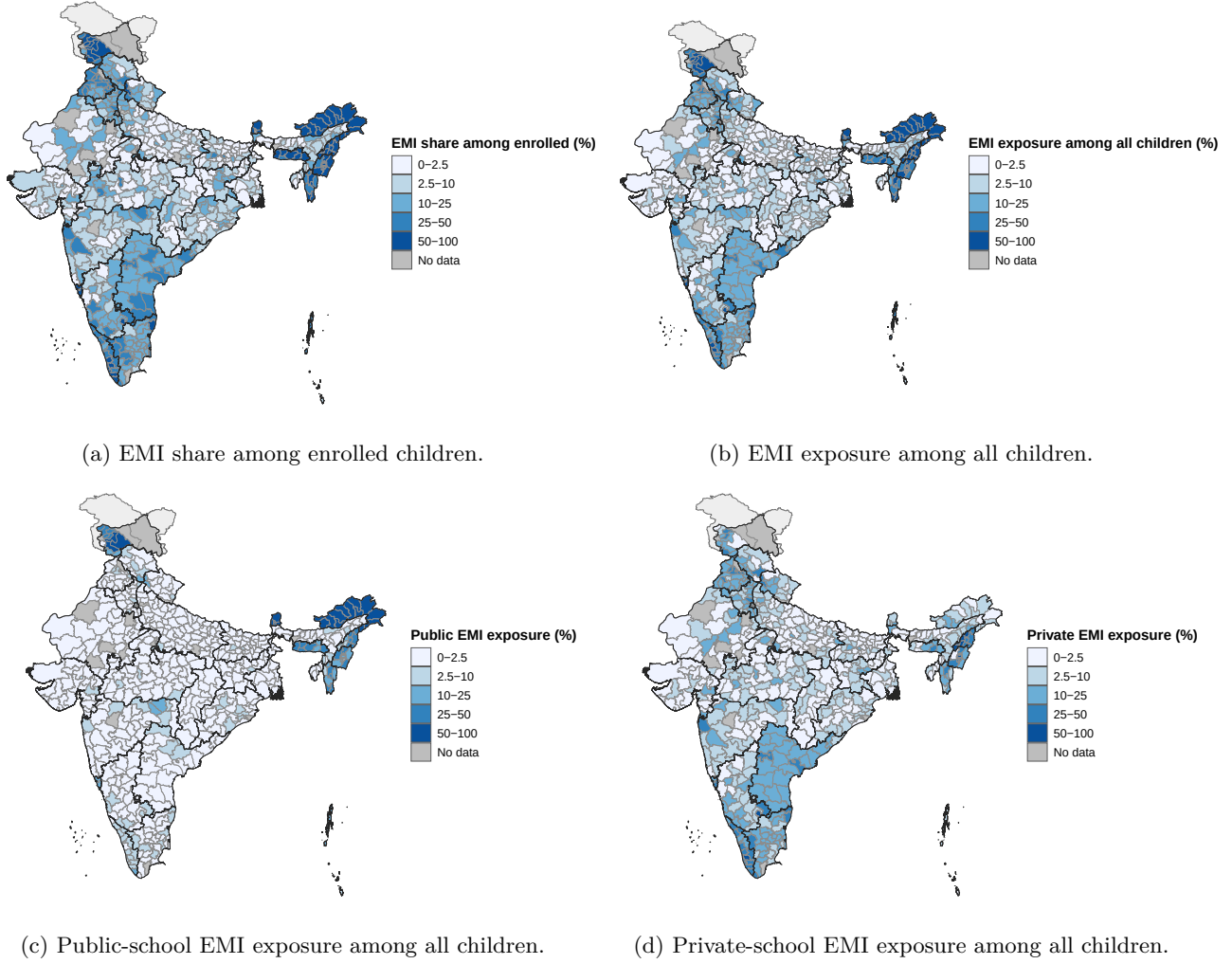


Figure 3: District distribution of English-medium schooling measures on 2001 Census boundaries. All panels use a common percentage scale. Source: NSS 64th Round, 2007–08 (National Sample Survey Office 2011). Tengshe (2014) and Natural Earth (2022) provide geospatial data. District boundaries reflect 2001 geography. Politically disputed regions are identified using Natural Earth, and those without data—Aksai Chin, Azad Kashmir, Gilgit-Baltistan, the Trans-Karakoram Tract, and Siachen Glacier—are colored light gray. Their inclusion does not imply a position on territorial disputes.

Linking Rural and Urban State CPI Series Across Index Bases

Old and new rural and urban state CPI series are linked over their January 2013–December 2014 overlap using the median monthly ratio of the new index to the old index. At least six common months are required, and the returned data retain the overlap counts and ratio dispersion used to assess sensitivity to the link period.

File: R/prices/build_temporal_price_series.R

```
build_temporal_price_series <- function(
  price_sources,
  switch_date = as.Date("2013-01-01"),
  overlap_start = as.Date("2013-01-01"),
  overlap_end = as.Date("2014-12-01"),
  minimum_link_months = 6L,
  pre_switch_start = NULL,
  pre_switch_end = NULL) {
  switch_date <- price_boundary(switch_date)
  old <- select_pre_2013_price_series(price_sources)
  new <- select_post_2013_price_series(price_sources)
  links <- summarise_price_links(old, new, overlap_start, overlap_end)

  minimum_link_months <- as.integer(minimum_link_months)
  if (length(minimum_link_months) != 1L || is.na(minimum_link_months) || minimum_link_months < 1L) {
    stop("minimum_link_months must be one positive integer.", call. = FALSE)
  }

  old_groups <- unique(old[old$period < switch_date, c("state_code", "sector"), drop = FALSE])
  linked_groups <- links[links$link_months >= minimum_link_months, c("state_code", "sector"), drop = FALSE]
  linked_key <- paste(linked_groups$state_code, linked_groups$sector, sep = "\r")
  missing_key <- !paste(old_groups$state_code, old_groups$sector, sep = "\r") %in% linked_key
  if (any(missing_key)) {
    bad <- old_groups[missing_key, , drop = FALSE]
    stop(
      "Pre-2013 price series lack a sufficient direct link to CPI-R/U: ",
      paste(paste(bad$state_code, bad$sector, sep = "/"), collapse = ", "),
      call. = FALSE
    )
  }

  old <- merge(old, links, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
  old <- old[old$period < switch_date, , drop = FALSE]
```

```

if (!is.null(pre_switch_start) || !is.null(pre_switch_end)) {
  if (is.null(pre_switch_start) || is.null(pre_switch_end)) {
    stop("Both pre_switch_start and pre_switch_end are required when trimming the old price series.",
      call. = FALSE)
  }
  pre_switch_start <- price_boundary(pre_switch_start)
  pre_switch_end <- price_boundary(pre_switch_end)
  if (pre_switch_end < pre_switch_start || pre_switch_end >= switch_date) {
    stop("The pre-switch production window must end before the switch date.", call. = FALSE)
  }
  old <- old[old$period >= pre_switch_start & old$period <= pre_switch_end, , drop = FALSE]
}
old$index_unlinked <- old$index
old$index <- num(old$index) * num(old$link_factor)

new <- new[new$period >= switch_date, , drop = FALSE]
new$link_factor <- 1
new$link_months <- NA_integer_
new$link_ratio_mad <- NA_real_
new$link_ratio_min <- NA_real_
new$link_ratio_max <- NA_real_
new$index_unlinked <- new$index

columns <- c(
  "state_code", "sector", "period", "index", "index_unlinked", "price_source",
  "link_factor", "link_months", "link_ratio_mad", "link_ratio_min", "link_ratio_max"
)
index <- rbind(old[columns], new[columns])
index$year <- as.integer(format(index$period, "%Y"))
index$month <- as.integer(format(index$period, "%m"))
index <- index[order(index$state_code, index$sector, index$period), , drop = FALSE]
rownames(index) <- NULL
validate_temporal_price_chain(index, switch_date)

structure(
  list(index = index, links = links, switch_date = switch_date),
  class = "emi_temporal_price_series"
)
}

```

Rural and Urban State Price Deflators for Comparable Real Consumption

The linked rural and urban CPI series is combined with state-specific price relatives for rural and urban areas derived from Tendulkar poverty lines. The resulting deflators express survey consumption in comparable 2011–12 rupees; the figure below compares district estimates across the 2022–23 and 2023–24 Household Consumption Expenditure Survey (HCES) rounds.

File: R/prices/price_deflators.R

```
build_state_sector_deflator <- function(
  temporal_index, spatial_relatives, reference_period = NULL, reference_index = NULL) {
  idx <- safe_df(temporal_index)
  spatial <- safe_df(spatial_relatives)
  validate_price_index(idx, keys = c("state_code", "sector", "period"))
  required_spatial <- c("state_code", "sector", "spatial_price_relative")
  missing <- setdiff(required_spatial, names(spatial))
  if (length(missing)) {
    stop("Spatial price relatives are missing columns: ", paste(missing, collapse = ", "), call. = FALSE)
  }
  if (anyDuplicated(spatial[c("state_code", "sector")])) {
    stop("Spatial price relatives must have one row per state and sector.", call. = FALSE)
  }
  if (any(!positive_finite(num(spatial$spatial_price_relative)))) {
    stop("Spatial price relatives must be positive and finite.", call. = FALSE)
  }

  if (is.null(reference_index)) {
    if (is.null(reference_period)) stop("A price reference period or reference-index table is required.",
    ↪ call. = FALSE)
    reference_period <- as.Date(reference_period)
    ref <- idx[idx$period %in% reference_period, , drop = FALSE]
    if (!nrow(ref)) stop("No observations fall in the requested price reference period.", call. = FALSE)
    ref_mean <- stats::aggregate(index ~ state_code + sector, ref, function(x) mean(num(x), na.rm = TRUE))
    names(ref_mean)[names(ref_mean) == "index"] <- "reference_index"
  } else {
    ref_mean <- safe_df(reference_index)
    required_reference <- c("state_code", "sector", "reference_index")
    missing_reference <- setdiff(required_reference, names(ref_mean))
    if (length(missing_reference)) {
      stop("Reference-price table is missing columns: ", paste(missing_reference, collapse = ", "), call. =
      ↪ FALSE)
    }
  }
}
```

```

}

if (anyDuplicated(ref_mean[c("state_code", "sector")]) ||
    ↪ any(!positive_finite(ref_mean$reference_index))) {
  stop("Reference-price table must have one positive row per state and rural/urban sector.", call. =
    ↪ FALSE)
}

ref_mean <- ref_mean[required_reference]
}

out <- merge(
  idx,
  spatial,
  by = c("state_code", "sector"), all.x = TRUE, sort = FALSE
)

out <- merge(out, ref_mean, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
if ("source_state_code" %in% names(out) && any(!positive_finite(out$reference_index))) {
  donor_reference <- ref_mean
  names(donor_reference)[names(donor_reference) == "state_code"] <- "source_state_code"
  names(donor_reference)[names(donor_reference) == "reference_index"] <- "donor_reference_index"
  out <- merge(out, donor_reference, by = c("source_state_code", "sector"), all.x = TRUE, sort = FALSE)
  missing_reference <- !positive_finite(out$reference_index)
  out$reference_index[missing_reference] <- out$donor_reference_index[missing_reference]
  out$donor_reference_index <- NULL
}

out$temporal_price_relative <- num(out$index) / num(out$reference_index)
out$price_deflator <- num(out$spatial_price_relative) * num(out$temporal_price_relative)
if (any(!positive_finite(out$price_deflator))) {
  stop("Deflator construction left missing or invalid values.", call. = FALSE)
}

out <- out[order(out$state_code, out$sector, out$period), , drop = FALSE]
rownames(out) <- NULL
out
}

build_state_sector_price_deflators <- function(
  temporal_series,
  state_rules = read_price_state_crosswalk(),
  poverty_lines = read_tendulkar_poverty_lines(),
  reference_period = seq(as.Date("2011-07-01"), as.Date("2012-06-01"), by = "month"),
  reference_index = NULL,
  start_period = NULL,
  end_period = NULL,

```

```

    reference_rupees = tendulkar_real_poverty_line(),
    require_complete_grid = TRUE) {
temporal_index <- if (inherits(temporal_series, "emi_temporal_price_series")) {
  temporal_series$index
} else {
  safe_df(temporal_series)
}
expanded <- apply_price_state_rules(
  temporal_index,
  state_rules,
  start_period = start_period,
  end_period = end_period,
  require_complete_grid = require_complete_grid
)
spatial <- build_tendulkar_spatial_relatives(poverty_lines, reference_rupees)
missing_spatial <- setdiff(
  unique(paste(expanded$state_code, expanded$sector, sep = "\r")),
  unique(paste(spatial$state_code, spatial$sector, sep = "\r"))
)
if (length(missing_spatial)) {
  stop("Tendulkar poverty-line data do not cover every state and rural/urban CPI series.", call. = FALSE)
}
build_state_sector_deflator(
  expanded,
  spatial,
  reference_period = reference_period,
  reference_index = reference_index
)
}

```

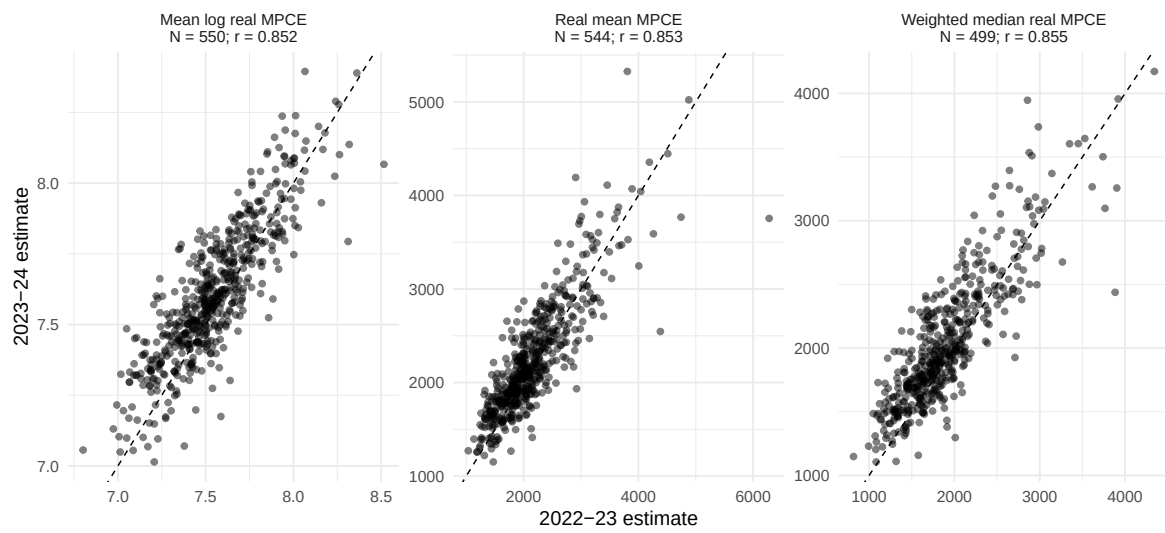


Figure 8: Agreement between district real-consumption estimates in the 2022-23 and 2023-24 HCES rounds. Each panel uses districts retained in both rounds under the consumption sample criteria; the dashed line is the 45-degree line. Sources: National Sample Survey Office, HCES 2022-23 and HCES 2023-24 (National Sample Survey Office 2024, 2025).